

Contents

Apuntes — Análisis raster con Python	1
1. ¿Qué es un raster?	1
2. El stack de librerías raster en Python	1
3. Conceptos clave de rasterio	2
4. La regla de oro de la álgebra de bandas	2
5. Índices espectrales — chuleta	3
6. Bandas Landsat 8/9 y Sentinel-2 — equivalencias	4
7. Reflectancia: qué es y por qué reescalar	4
8. Visualizar bandas y composiciones RGB	5
9. Guardar un raster derivado	5
10. Raster ↔ vector	6
11. Errores típicos y cómo evitarlos	6
12. GDAL CLI — recetas que vale la pena memorizar	7
13. Recursos	7
14. Chuleta súper-resumen (la nevera)	8

Apuntes — Análisis raster con Python

Guía rápida para acompañar los notebooks **03a** (NumPy) y **03b** (Rasterio) del Día 3 del curso GeoPython. Pensada como **chuleta de referencia**: no sustituye al notebook, lo complementa.

1. ¿Qué es un raster?

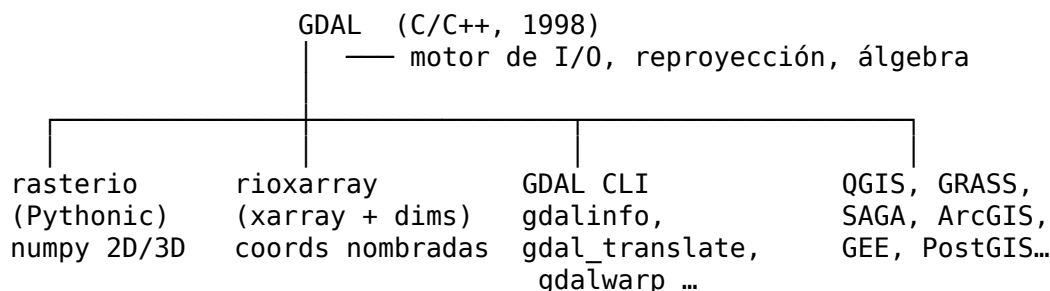
Un **raster** es una imagen georreferenciada: una cuadrícula regular de píxeles, donde cada píxel guarda **uno o varios valores** y la posición geográfica del píxel se deduce de:

- el **CRS** (sistema de referencia, p. ej. EPSG:25830 = ETRS89 UTM 30N),
- el **transform**: tamaño de píxel y coordenadas de la esquina superior izquierda.

Ejemplos: imagen Landsat o Sentinel-2, MDT, mapa de pendientes, índice NDVI, máscara de inundación.

Un raster **multibanda** es simplemente un montón de cuadrículas apiladas (varias bandas espectrales, o varias fechas, o varias variables).

2. El stack de librerías raster en Python



- **GDAL** lo entiende todo y lo convierte todo. Pesado pero robusto. Su CLI (gdalinfo, gdal_translate, gdalwarp) es el “todoterreno” del SIG.
- **rasterio** = API Pythonica encima de GDAL. Te devuelve **arrays NumPy** sin más adornos.

- **rioxarray** = lo mismo pero con coordenadas/dimensiones nombradas (x, y, band, time). Útil para series temporales.
- **rasterstats** = estadísticas zonales (raster + vector → tabla).
- **NumPy** = el lenguaje en el que se escribe la teledetección. **Imprescindible**.

3. Conceptos clave de rasterio

Concepto	Qué es
<code>src = rasterio.open(path)</code>	Abre el raster sin leer los píxeles . Solo metadatos.
<code>src.crs</code>	Sistema de coordenadas (CRS)
<code>src.transform</code>	Affine que mapea (col, fila) → (x, y) del mundo
<code>src.bounds</code>	(left, bottom, right, top)
<code>src.width, src.height</code>	Dimensiones en píxeles
<code>src.count</code>	Nº de bandas
<code>src.dtypes</code>	Tipo de dato de cada banda
<code>src.profile</code>	Dict con todos los metadatos. Lo usaremos al escribir
<code>src.read(n)</code>	Lee la banda n (numerada desde 1) → array 2D
<code>src.read()</code>	Lee todas las bandas → array 3D (bandas, alto, ancho)
<code>src.close()</code>	Liberar el fichero. Mejor usar with <code>rasterio.open(...)</code> as <code>src</code> :

Affine transform al detalle

`Affine(a, b, c,
 d, e, f)`

- a = tamaño de píxel en X (positivo)
- e = tamaño de píxel en Y (**negativo**, porque los rasters se almacenan de arriba abajo)
- c, f = coordenadas (x, y) de la **esquina superior izquierda**
- b, d = rotación (casi siempre 0)

Conversiones útiles:

```
x, y      = src.transform * (col, fila)  # píxel → coordenadas
fila, col  = src.index(x, y)             # coordenadas → píxel
```

4. La regla de oro de la álgebra de bandas

Antes de dividir bandas (NDVI, NDWI, etc.) **convierte siempre a float**:

```
nir = src.read(4).astype('float32')
red = src.read(3).astype('float32')
ndvi = (nir - red) / (nir + red)
```

Si lo haces sin `astype`, las bandas enteras (típicas `uint16`) **truncan las divisiones a 0** y el NDVI sale plano. Es el error nº 1 de los recién llegados.

Suprimir warnings de división por cero

En los píxeles donde NIR y Red son ambos 0 (nodata, sombra...), la división peta. La forma elegante:

```
import numpy as np
with np.errstate(divide='ignore', invalid='ignore'):
    ndvi = (nir - red) / (nir + red)
```

5. Índices espectrales — chuleta

Índice	Fórmula	Rango típico	Para qué
NDVI	$(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$	-1 a 1	Vigor vegetal. Veg sana > 0.6; suelo ~0.1-0.3; agua ≤ 0
EVI	$2.5 \cdot (\text{NIR} - \text{Red}) / (\text{NIR} + 6 \cdot \text{Red} - 7.5 \cdot \text{Blue} + 1)$	-1 a 1	NDVI mejorado: saturación menor en biomasa alta
NDWI (McFeeters 1996)	$(\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR})$	-1 a 1	Agua. Positivo = agua
MNDWI (Xu 2006)	$(\text{Green} - \text{SWIR1}) / (\text{Green} + \text{SWIR1})$	-1 a 1	Agua, mejor que NDWI en marismas y aguas turbias
NDMI	$(\text{NIR} - \text{SWIR1}) / (\text{NIR} + \text{SWIR1})$	-1 a 1	Humedad del dosel vegetal
NBR	$(\text{NIR} - \text{SWIR2}) / (\text{NIR} + \text{SWIR2})$	-1 a 1	Áreas quemadas
CI green	$\text{NIR}/\text{Green} - 1$	≥ 0	Proxy de clorofila en vegetación y agua
NDCI	$(\text{RedEdge} - \text{Red}) / (\text{RedEdge} + \text{Red})$	-1 a 1	Clorofila en agua (solo Sentinel-2: necesita Red Edge)

Truco mnemotécnico: los índices “ND...” se construyen con $(A - B) / (A + B)$. Es la forma normalizada estándar. La fórmula resta el “fondo” y suma para acotar al rango [-1, 1].

Filtrado de nubes y sombras antes de calcular índices

Las nubes saturan el NDVI a ~0 y las sombras lo bajan artificialmente. **Antes de calcular índices o estadísticas zonales sobre datos reales, filtra siempre por la máscara de calidad** que viene con cada producto:

Sensor	Banda de calidad	Cómo se interpreta
Landsat 8/9 C2 L2	QA_PIXEL (uint16)	Códigos enteros (21824, 21952...) — ver abajo
Sentinel-2 L2A	SCL (uint8)	Códigos 0-11 (4=veg, 5=suelo, 6=agua, 8/9=nube, 3=sombra)
MODIS, VIIRS	QA o state_1km	Bits propios de cada producto

Filtro práctico para Landsat 8/9 C2 — los códigos que nos importan en una escena típica:

Código	Significado
21824	terreno claro
21952	agua clara
22280	nube
23888	sombra de nube
55052	nube + cirrus
56660	sombra + cirrus

```
qa = src.read(7).astype('int32')
pixel_bueno = (qa == 21824) | (qa == 21952) # terreno claro o agua clara
ndvi_limpio = np.where(pixel_bueno, ndvi, np.nan)
```

De dónde salen estos números (lectura avanzada, no imprescindible): USGS empaqueta varias condiciones (nube, sombra, cirrus, agua, nieve, confianzas...) en los **bits** de un entero de 16 bits. Cada valor es la suma de bits encendidos: 21824 = 0101010101000000₂ significa “clear=1, low confidence en nube/sombra/snow/cirrus”. Si quieres extraer una flag concreta (p. ej. “es nube”) sin enumerar todos los valores, se hace con desplazamiento de bits: `is_cloud = (qa >> 3) & 1`. La operación `>> N` corre los bits N posiciones a la derecha y `& 1` se queda con el bit más bajo. Es la forma canónica de “extraer una flag” empaquetada en un entero. Para una escena concreta basta con filtrar por igualdad como arriba.

6. Bandas Landsat 8/9 y Sentinel-2 — equivalencias

Función	Landsat 8/9 (30 m)	Sentinel-2 (10/20 m)
Coastal	B1	B1 (60 m)
Blue	B2	B2 (10 m)
Green	B3	B3 (10 m)
Red	B4	B4 (10 m)
Red Edge 1	—	B5 (20 m)
Red Edge 2	—	B6 (20 m)
Red Edge 3	—	B7 (20 m)
NIR	B5	B8 (10 m) ó B8A (20 m)
Water vapor	—	B9 (60 m)
Cirrus	B9	B10
SWIR1	B6	B11 (20 m)
SWIR2	B7	B12 (20 m)
TIR	B10, B11 (térmico)	— (S2 no térmico)

Sentinel-2 tiene **Red Edge** — claves para clorofila en cultivos y blooms en agua. Landsat tiene **térmico** — clave para temperatura superficial.

7. Reflectancia: qué es y por qué reescalar

Los satélites no graban reflectancia directamente: graban **niveles digitales (DN)** que después USGS/ESA convierten a reflectancia mediante factores de escala.

Landsat Collection 2 Level-2 (Surface Reflectance)

```
reflectancia = DN * 0.0000275 - 0.2
```

Una banda viene como uint16. Tras aplicar la escala queda en float32 y el rango razonable es **[0, 1]** (valores fuera = ruido o nube).

Sentinel-2 L2A

reflectancia = DN / 10000 # valores 0–1 (o 0–1.4 con nubes)

Si te dan la imagen **ya reescalada a float [0, 1]** (como en este curso) → te puedes saltar este paso. Si te dan la “cruda” de USGS, **aplica la escala antes de calcular índices**.

8. Visualizar bandas y composiciones RGB

matplotlib.pyplot.imshow espera (**alto, ancho, 3**) para RGB. Rasterio entrega (**bandas, alto, ancho**). Convierte con np.dstack o np.moveaxis(arr, 0, -1).

```
import numpy as np
rgb = np.dstack([estirar(red), estirar(green), estirar(blue)])
plt.imshow(rgb)
```

Estiramiento por percentiles

Las reflectancias son chicas (típicamente 0.05–0.3). Sin estirar, todo sale gris oscuro:

```
def estirar(banda, p_min=2, p_max=98):
    lo, hi = np.percentile(banda, [p_min, p_max])
    return np.clip((banda - lo) / (hi - lo), 0, 1)
```

Composiciones típicas

Nombre	R · G · B	Útil para
Color natural	Red · Green · Blue	“Lo que ven los ojos”;
Falso color IR	NIR · Red · Green	cultivos
Agro / SWIR	SWIR1 · NIR · Red	Vegetación en rojo intenso
Quemados	SWIR2 · NIR · Green	Cultivos vs suelo, humedad
		Áreas calcinadas en marrón

9. Guardar un raster derivado

Receta universal: parte del profile del raster original y modifica lo que cambia.

```
profile = src.profile.copy()
profile.update(
    count=1,                      # solo NDVI = 1 banda
    dtype='float32',
    nodata=np.nan,
    compress='deflate', # 3–5× más ligero
    tiled=True,                  # bloques 256×256 → casi-COG
)
with rasterio.open('ndvi.tif', 'w', **profile) as dst:
```

```
dst.write(ndvi.astype('float32'), 1)
dst.set_band_description(1, 'NDVI')
```

Siempre compress='deflate' salvo que tengas un motivo en contra. Reduce el tamaño sin pérdida.

10. Raster ↔ vector

Recortar un raster por un polígono

```
from rasterio.mask import mask
geom = [poligono.__geo_interface__]
img, transform = mask(src, geom, crop=True, nodata=np.nan, filled=True)
```

Polígonos a partir de una máscara binaria

```
from rasterio.features import shapes
from shapely.geometry import shape
import geopandas as gpd

mascara = (mndwi > 0.1).astype('uint8')
geoms = [
    {'geometry': shape(g), 'val': v}
    for g, v in shapes(mascara, mask=(mascara==1), transform=src.transform)
]
gdf = gpd.GeoDataFrame(geoms, crs=src.crs)
```

“Quemar” un vectorial sobre un raster (rasterize)

```
from rasterio.features import rasterize
arr = rasterize(
    [(geom, 1) for geom in mi_gdf.geometry],
    out_shape=(src.height, src.width),
    transform=src.transform,
    fill=0, dtype='uint8',
)
```

Estadísticas zonales

```
from rasterstats import zonal_stats
stats = zonal_stats(
    'municipios.gpkg', 'ndvi.tif',
    stats=['mean', 'std', 'min', 'max', 'count'],
    nodata=np.nan,
)
```

11. Errores típicos y cómo evitarlos

1. **NDVI sale plano (todo a 0)** → has dividido enteros. Solución: `astype('float32')`.
2. **El recorte por vector falla con “CRS mismatch”** → reprojeta el vector al CRS del raster: `gdf.to_crs(src.crs)`.
3. **imshow no muestra colores en una RGB** → la forma es (3, alto, ancho) en vez de (alto, ancho, 3). Usa `np.dstack` o `np.moveaxis(arr, 0, -1)`.

4. **El raster sale negro** → reflectancias muy bajas, hace falta estirar por percentiles (`vmin=p2`, `vmax=p98`).
 5. **Memoria agotada en Colab** → no leas la escena entera. Usa `src.read(window=Window(col, row, w, h))` o trabaja con un recorte.
 6. **NoData no se propaga** → en NumPy, los nan necesitan `np.nanmean`, `np.nanmax`, etc. Si abres con `nodata=0`, rasterio crea un `MaskedArray`.
 7. **Las bandas Sentinel tienen distinta resolución** → necesitas **remuestrear** B11/B12 (20 m) a 10 m antes de mezclarlas con B8 (10 m). Usa `rasterio.warp.reproject` o `rioxarray.reproject_match`.
 8. **gdalinfo muestra "WGS84" pero el píxel está en metros** → seguramente es Pseudo-Mercator (EPSG:3857). No es lo mismo que UTM.
-

12. GDAL CLI — recetas que vale la pena memorizar

Inspección rápida

```
gdalinfo escena.tif
```

```
gdalinfo -stats escena.tif          # incluye min/max/media/std por banda
```

Extraer una banda

```
gdal_translate -b 4 escena.tif solo_nir.tif
```

Convertir a COG (Cloud Optimized GeoTIFF)

```
gdal_translate escena.tif escena_cog.tif \
    -co COMPRESS=DEFLATE -co TILED=YES \
    -co BLOCKXSIZE=512 -co BLOCKYSIZE=512 \
    -co COPY_SRC_OVERVIEWS=YES
```

Reproyectar

```
gdalwarp -t_srs EPSG:25830 -r bilinear in.tif out.tif
```

Recortar por bbox

```
gdalwarp -te 175000 4080000 220000 4140000 in.tif recorte.tif
```

Recortar por vectorial

```
gdalwarp -cutline almonte.gpkg -crop_to_cutline in.tif almonte.tif
```

Crear mosaico virtual de varias escenas (no copia bytes, solo enlaza)

```
gdalbuildvrt mosaico.vrt escena_1.tif escena_2.tif escena_3.tif
```

13. Recursos

- **Documentación rasterio**: <https://rasterio.readthedocs.io>
 - **Documentación GDAL CLI**: <https://gdal.org/programs/index.html>
 - **Spatial Thoughts — Python Foundation for Spatial Analysis**: <https://courses.spatialthoughts.com/python-foundation.html> (gratis, excelente)
 - **USGS EarthExplorer** (descarga Landsat): <https://earthexplorer.usgs.gov>
 - **Copernicus Browser** (descarga Sentinel): <https://browser.dataspace.copernicus.eu>
 - **GEE Code Editor** (lo veremos en el módulo siguiente): <https://code.earthengine.google.com>
 - **Pyrosm / Pystac / Planetary Computer**: para acceso a catálogos abiertos por API.
-

14. Chuleta súper-resumen (la nevera)

```
import rasterio, numpy as np, geopandas as gpd
import matplotlib.pyplot as plt

# Abrir y leer
src = rasterio.open('landsat.tif')
red, nir = src.read(3).astype('f4'), src.read(4).astype('f4')

# Índice
with np.errstate(divide='ignore', invalid='ignore'):
    ndvi = (nir - red) / (nir + red)

# Visualizar
plt.imshow(ndvi, cmap='RdYlGn', vmin=-0.2, vmax=0.8); plt.colorbar(); plt.show()

# Guardar
profile = src.profile.copy()
profile.update(count=1, dtype='float32', compress='deflate', nodata=np.nan)
with rasterio.open('ndvi.tif', 'w', **profile) as dst:
    dst.write(ndvi, 1)

# Máscara de agua → polígonos
from rasterio.features import shapes
from shapely.geometry import shape
agua = ((nir - src.read(2).astype('f4'))/(nir + src.read(2).astype('f4')) < -0.1).astype('u1')
polys = [shape(g) for g, v in shapes(agua, mask=(agua==1), transform=src.transform)]
gpd.GeoDataFrame(geometry=polys, crs=src.crs).to_file('agua.gpkg', driver='GPKG')

# Estadísticas zonales
from rasterstats import zonal_stats
stats = zonal_stats('municipios.gpkg', 'ndvi.tif', stats=['mean', 'std', 'count'])
```

CHG — Curso de Python para el Análisis Espacial. Día 3.